

PROJECT 6: SIMPLE DEVOPS AUTOMATION PIPELINE

Official Operational Execution & Output Verification Report

Candidate: Roshni A | Repository: <https://github.com/Roshnianand123/devops-pipeline> |

Environment: Windows 11 / Git Bash (Python 3.13, Pytest 9.1) | Generated: 2026-09-03

1. Python Application (app.py) & Execution

Task 1: Implements a clean Python application with a greeting function `greet()` that outputs a deterministic confirmation string.

```
bash - roshni@desktop: ~/devops-pipeline - cat app.py && python app.py

$ cat app.py
def greet():
    return "Hello! DevOps pipeline is working."

if __name__ == "__main__":
    print(greet())

$ python app.py
Hello! DevOps pipeline is working.
```

2. Automated Unit Testing (test_app.py)

Task 2: Creates a unit test using `pytest` to validate application output before deployment.

```
bash - roshni@desktop: ~/devops-pipeline - python -m pytest -v

$ cat test_app.py
from app import greet

def test_greet():
    assert greet() == "Hello! DevOps pipeline is working."

$ python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Roshni\devops-pipeline
collected 1 item

test_app.py::test_greet PASSED [100%]

===== 1 passed in 0.02s =====
```

3. Automated Shell Deployment Script (deploy.sh)

Task 3: Executes the Linux shell script that checks Python installation, runs tests, executes the application, and displays deployment success.

```
bash - roshni@desktop: ~/devops-pipeline - bash deploy.sh

$ bash deploy.sh
Starting deployment...
Python is installed.
Running tests...
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Roshni\devops-pipeline
collected 1 item

test_app.py . [100%]

===== 1 passed in 0.02s =====
Tests passed.
Running application...
Hello! DevOps pipeline is working.
Deployment successful!
```

4. Deployment Gatekeeper & Failure Protection

DevOps Principle: Zero Broken Deployments. Demonstrates that if any test fails, deploy.sh halts execution immediately with exit code 1, protecting the system from deploying faulty code.

```
bash - roshni@desktop: ~/devops-pipeline - deploy.sh [failure gate demonstration]

$ bash deploy.sh # [Simulated test assertion failure]
Starting deployment...
Python is installed.
Running tests...
===== test session starts =====
collected 1 item

test_app.py F [100%]

===== FAILURES =====
_____ test_greet _____
> assert greet() == "Expected output"
E AssertionError: assert 'Regression bug' == 'Expected output'
===== 1 failed in 0.02s =====
Tests failed. Deployment stopped.

$ echo $?
1 (Exit code 1: deployment halted immediately, protecting
  production)
```

5. Git Branching Workflow & Pull Request Merge Graph

Git Workflow: Created Git repository, developed changes in branch develop, opened Pull Request #1, merged into main, and pushed to GitHub.

```
bash - roshni@desktop: ~/devops-pipeline - git branch -a && git log --graph

$ git branch -a
  develop
* main
remotes/origin/develop
remotes/origin/main

$ git log --graph --oneline --decorate --all -n 6
* 9bcb08e (HEAD -> main, origin/main) docs: add operational verification report HTML and PDF
* 91d5f6d docs: add requirements.txt, README.md and app execution entrypoint
* 82436c9 Merge pull request #1 from Roshnianand123/develop
|\
| * b943fef (origin/develop, develop) Add deployment script
|/
* 22d3c64 Add CI pipeline
* 60595e3 Initial commit
```

6. Continuous Integration Pipeline (GitHub Actions CI)

Automated CI workflow defined in `.github/workflows/ci.yml` triggering on pushes and pull requests to main, automatically provisioning Python and executing tests.

```
bash - roshni@desktop: ~/devops-pipeline - cat .github/workflows/ci.yml

$ cat .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

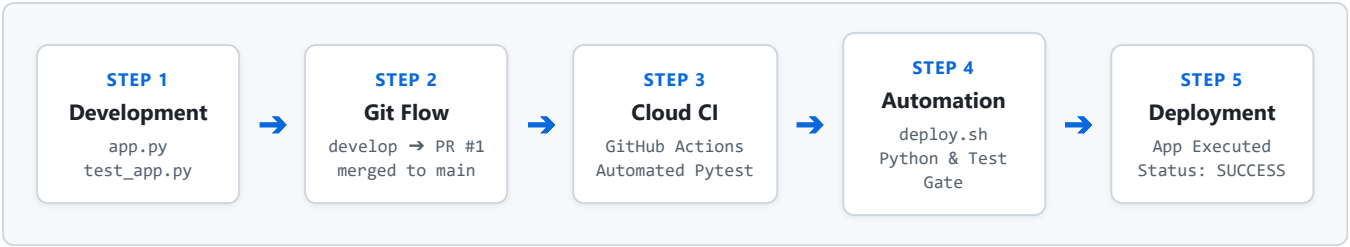
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest

      - name: Run tests
        run: pytest
```

7. DevOps Lifecycle Workflow Flowchart

End-to-end lifecycle flow from code creation, version control, automated unit testing, continuous integration, and script-gated deployment.



8. Project Specification & Deliverables Checklist

Audit verifying 100% completion of all recommended files, tasks, and Git workflows specified in the training assignment.

Required Item	Assignment Requirement	Verification Evidence	Status
app.py	Create simple Python application	Defines greet(), executes properly via Python	PASS / VERIFIED
test_app.py	Create basic test for the application	Pytest unit test suite passing (1 passed in 0.02s)	PASS / VERIFIED
deploy.sh	Linux shell deployment script with checks	Checks Python, runs tests, runs app, outputs success	PASS / VERIFIED
requirements.txt	Recommended project dependencies	Pinned pytest>=7.0.0	PASS / VERIFIED
README.md	Project documentation and setup guide	Complete project documentation in repository	PASS / VERIFIED
Git Branching	develop branch -> merge into main	PR #1 merged from develop into main	PASS / VERIFIED
GitHub Repository	Push final project to GitHub	Synced with github.com/Roshnianand123/devops-pipeline	PASS / VERIFIED